

**Методичні вказівки до виконання лабораторних робіт
з дисципліни: Поглиблене вивчення JavaScript**

**Лабораторна робота №2
Розробка галереї зображень із використанням API**

2.1 Мета роботи

Ознайомлення з механізмами асинхронного програмування в JavaScript, опанування способів взаємодії з зовнішніми API засобами Vue 3, формування практичних навичок обробки асинхронних даних, станів завантаження та помилок у сучасних SPA-застосунках.

2.2 Загальні відомості

Більшість сучасних вебзастосунків взаємодіють із зовнішніми джерелами даних через мережеві запити. Такий підхід дозволяє динамічно отримувати актуальну інформацію з серверів та відображати її у користувацькому інтерфейсі без перезавантаження сторінки.

У межах даної лабораторної роботи реалізується галерея зображень, яка завантажує дані з відкритого API — в методичних вказівках розглядається Picsum Photos. Це дозволяє відпрацювати типовий сценарій роботи фронтенд-розробника: від формування запиту до відображення результатів та обробки помилок.

Застосовані у цій роботі підходи є базовими для будь-якого фронтенд-проєкту, що взаємодіє із серверною частиною.

2.3 Теоретичні відомості

Асинхронне програмування є ключовою концепцією сучасного JavaScript. Оскільки мережеві запити вимагають часу на виконання, браузер не може чекати їх завершення і блокувати інтерфейс. Для зручної роботи з такими операціями у JavaScript існує поняття Promise — об'єкт, що представляє результат операції, яка ще не завершилась. Promise може перебувати в одному з трьох станів: очікування (pending), успішне завершення (fulfilled) або помилка (rejected).

Синтаксис `async/await` є сучасним способом роботи з асинхронними операціями в JavaScript. Функція завжди повертає Promise, навіть якщо в тілі немає явного повернення об'єкта. Всередині `async`-функції можна використовувати ключове слово `await` перед будь-яким Promise — виконання функції припиняється до отримання результату, а потім продовжиться далі. Це дозволяє писати асинхронний код у лінійному стилі, без вкладених зворотних викликів, що значно покращує читабельність.

Для виконання HTTP-запитів використовується вбудований у браузер метод `fetch`. Він приймає URL-адресу ресурсу та необов'язковий об'єкт налаштувань (метод запиту, заголовки, тіло). Метод повертає Promise, який після отримання відповіді від сервера переходить у стан `fulfilled` зі спеціальним об'єктом `Response`. Важливо розуміти, що `fetch` вважає успішним будь-який запит, який отримав відповідь від сервера — навіть із кодом помилки 404 або 500. Перевіряти успішність слід самостійно через властивість `response.ok`. Для отримання даних у форматі JSON необхідно викликати метод `response.json()`, який також є асинхронним. Як альтернативу `fetch` можна використовувати

бібліотеку `axios`: вона автоматично перетворює відповідь у JSON, викидає виняток при кодах помилок і має більш зручний синтаксис для складних запитів.

Обробка помилок є обов'язковою частиною роботи з асинхронним кодом. Конструкція `try/catch` дозволяє перехопити помилки мережевого запиту, тайм-ауту або некоректні відповіді сервера та повідомити про них користувача. Додатково існує блок `finally`, що виконується завжди — як при успіху, так і при помилці. Він зручний для скидання стану завантаження незалежно від результату запиту.

Стан завантаження (`loading state`) — це окрема реактивна змінна, що сигналізує про те, що застосунок очікує відповідь від сервера. Протягом цього часу корисно відображати у інтерфейсі індикатор завантаження, щоб користувач розумів, що процес відбувається. Для цього використовується окрема реактивна булева змінна, яка встановлюється у `true` перед запитом і скидається у `false` після його завершення.

Хук `onMounted` є частиною системи життєвого циклу компонентів у Vue 3. Кожен компонент проходить кілька етапів існування: створення, монтування у DOM, оновлення при зміні даних та знищення. Vue надає набір хуків — функцій, що викликаються автоматично на певному етапі. Хук `onMounted` спрацьовує після того, як компонент повністю вставлено у DOM і всі дочірні компоненти також змонтовано. Саме тут рекомендується ініціювати завантаження початкових даних: на момент виклику гарантовано існують усі елементи розмітки, а користувач вже бачить інтерфейс. Хуки підключаються у блоці `script setup` шляхом виклику відповідної функції з передачею колбеку.

Обчислювані властивості (`computed`) у Vue 3 дозволяють формувати похідні дані на основі реактивних змінних. Вони автоматично перераховуються при зміні залежностей. Використання `computed` для фільтрації та пошуку є рекомендованим підходом завдяки вбудованому кешуванню.

2.4 Порядок виконання роботи

Виконання лабораторної роботи здійснюється у такій послідовності:

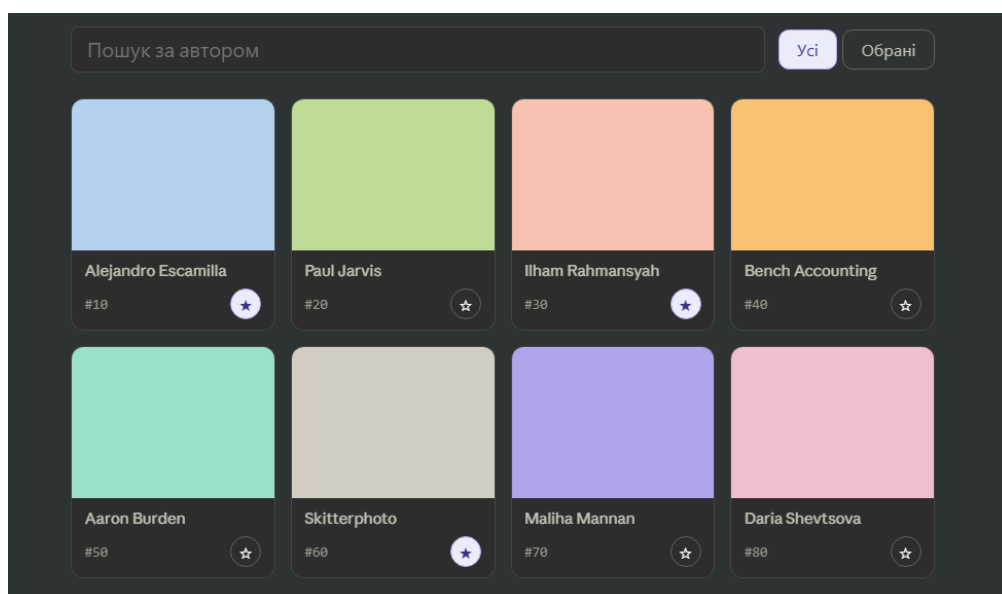
1. Підготувати проєкт: створити новий Vue-проєкт або продовжити роботу в наявному.
2. Ознайомитися з API Picsum Photos (<https://picsum.photos/>):
 - a. перейти за адресою <https://picsum.photos/v2/list?page=1&limit=20>
 - b. вивчити структуру відповіді: масив об'єктів із полями `id`, `author`, `download_url`
 - c. визначити, які поля використовуватимуться в інтерфейсі
3. Підготувати змінні стану в кореновому компоненті або окремому компоненті галереї:
 - a. масив зображень (`images`)
 - b. прапорець стану завантаження (`isLoading`)
 - c. змінну для збереження помилки (`error`)
 - d. масив обраних зображень (`favorites`)
 - e. рядок пошуку (`searchQuery`)
4. Реалізувати функцію завантаження даних:
 - a. використати `async/await` та `fetch`
 - b. встановлювати `isLoading` у `true` перед запитом та `false` після
 - c. зберігати помилки у відповідну змінну

5. Викликати функцію завантаження у хуку onMounted.
6. Реалізувати відображення галереї зображень:
 - a. використати v-for для ітерації по масиву зображень
 - b. відобразити зображення з підписом автора
 - c. забезпечити унікальний ключ для кожного елемента
7. Реалізувати стан завантаження:
 - a. відображати індикатор або текст «Завантаження...» під час отримання даних
 - b. приховувати галерею до завершення завантаження
8. Реалізувати відображення помилок:
 - a. показувати повідомлення про помилку при її наявності
 - b. передбачити можливість повторної спроби завантаження
9. Реалізувати функціонал обраного (favorites):
 - a. додати кнопку на кожній картці зображення
 - b. реалізувати логіку додавання та видалення з обраних
 - c. забезпечити відповідне візуальне відображення стану кнопки
10. Реалізувати фільтрацію за допомогою computed:
 - a. пошук за ім'ям автора через рядок введення
 - b. перемикання між режимами «Усі» та «Обрані»
11. Перевірити працездатність застосунку:
 - a. коректне завантаження та відображення зображень
 - b. відображення індикатора завантаження
 - c. коректна робота обраних та фільтрації
 - d. відображення помилки при недоступному API
 - e. відсутність помилок у консолі браузера

Додатково (за бажанням студента) реалізувати пагінацію або нескінченне прокручування через зміну параметра page в запиті. Виділити окремий компонент картки зображення.

2.5 Приклад реалізації застосунку

Реалізація застосунку базується на виклику зовнішнього API у хуку onMounted та відображенні отриманих даних у вигляді сітки карток.



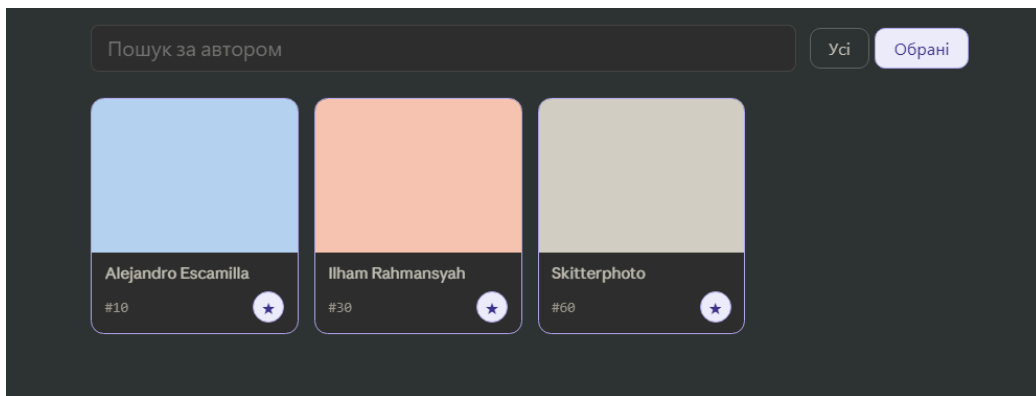
Для завантаження зображень використовується відкрите API Picsum Photos, яке не потребує реєстрації та автентифікації. Відповідь являє собою масив об'єктів, кожен з яких містить унікальний ідентифікатор, ім'я автора та URL зображення. Студент самостійно визначає, які поля використовувати та як відобразити їх в інтерфейсі.

Стан застосунку організовується через кілька реактивних змінних: масив отриманих зображень, прапорець стану завантаження та змінна для зберігання повідомлення про помилку. Завдяки реактивності Vue інтерфейс автоматично оновлюється при зміні будь-якої з цих змінних.

Функція завантаження даних викликається одразу після монтування компонента. Вона встановлює прапорець завантаження, виконує запит до API, зберігає результат у відповідну змінну та у разі помилки фіксує її для відображення користувачу. Прапорець завантаження скидається після завершення операції незалежно від її результату.

Фільтрація та пошук реалізуються через обчислювану властивість, яка формує похідний масив на основі поточного рядка пошуку та обраного режиму відображення. Список перераховується автоматично при зміні будь-якого з параметрів.

Функціонал «Обраних» реалізується на рівні клієнта: окремий масив зберігає ідентифікатори зображень, які користувач додав до обраних. Додавання та видалення відбуваються через перевірку наявності ідентифікатора у цьому масиві.



Шаблон компонента організовується за принципом умовного відображення: залежно від поточного стану показується або індикатор завантаження, або повідомлення про помилку, або сітка карток із зображеннями.

2.6 Завдання до виконання

У процесі виконання лабораторної роботи необхідно:

- підключитися до зовнішнього API та завантажити список зображень
- відобразити зображення у вигляді сітки карток із підписом автора
- реалізувати стан завантаження з індикатором
- реалізувати обробку та відображення помилок запиту
- забезпечити можливість додавання зображень до обраних та їх видалення
- реалізувати пошук за іменем автора через обчислювану властивість
- реалізувати фільтрацію між режимами «Усі» та «Обрані»

2.7 Вимоги до захисту

Усі лабораторні роботи виконуються у вигляді окремих проєктів або модулів одного проєкту. Код має бути структурованим, із чітким поділом на компоненти та дотриманням принципів чистого коду.

Результати роботи студент повинен представити у вигляді:

- вихідного коду
- короткого звіту з описом мети, теоретичних відомостей та реалізації
- демонстрації працездатності застосунку

Під час захисту особлива увага приділяється розумінню механізмів асинхронного програмування, логіці організації запитів та обробки відповідей, а також умінню пояснити власний код.

2.8 Оцінювання лабораторної роботи

Лабораторна робота оцінюється максимально у 25 балів. Оцінка залежить від повноти виконання завдання, працездатності застосунку та здатності студента пояснити свою реалізацію під час захисту.

Чим повніше реалізовано функціонал і стабільніше працює додаток, тим вищий бал. Частково виконані або некоректні роботи оцінюються нижче.

2.9 Контрольні питання

1. Що таке асинхронне програмування і чому воно необхідне у вебзастосунках?
2. Яке призначення ключових слів `async` та `await`?
3. Як виконати HTTP-запит за допомогою `fetch`? Яку структуру має відповідь?
4. Навіщо використовується хук `onMounted`? На якому етапі він виконується?
5. Що таке стан завантаження (`loading state`) і як його реалізувати у `Vue`?
6. Як обробити помилку мережевого запиту за допомогою `try/catch`?
7. Чим `computed` відрізняється від звичайної функції у `Vue`?
8. Як реалізувати локальний список обраних без збереження на сервері?